Bài tập – Chat Server

Sử dụng hàm select()/poll(), viết chương trình **chat_server** thực hiện các chức năng sau:

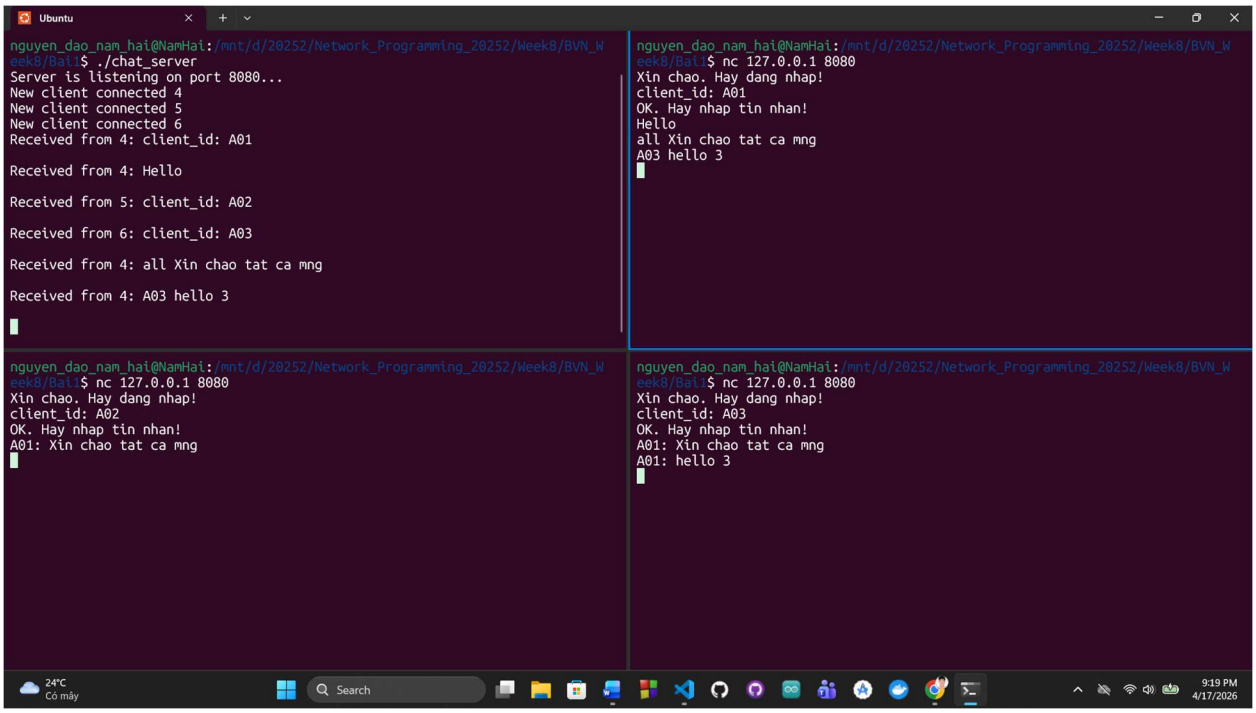
Nhận kết nối từ các client, và vào hỏi tên client cho đến khi client gửi đúng cú pháp:

“client_id: client_name”

trong đó client_name là tên của client, xâu ký tự viết liền.

Sau đó nhận dữ liệu từ một client và gửi dữ liệu đó đến các client còn lại, ví dụ: client có id “abc” gửi “xin chao” thì các client khác sẽ nhận được: “abc: xin chao” hoặc có thể thêm thời gian vào trước ví dụ: “2023/05/06 11:00:00PM abc: xin chao”.

Demo:



Giải thích:

Ô trên cùng bên trái chạy Server (quản lý kết nối). Ba ô còn lại đóng vai trò là 3 Client riêng biệt kết nối đến Server thông qua Netcat (nc 127.0.0.1 8080).

Đăng nhập và định danh:

Cả 3 Client đều kết nối thành công (Server ghi nhận FD 4, 5, 6) và đăng ký định danh hợp lệ với cú pháp chuẩn, lần lượt là A01 (Góc trên phải), A02 (Góc dưới trái) và A03 (Góc dưới phải).

Kiểm chứng tính năng Broadcast (Gửi tất cả): * Client A01 gửi lệnh: all Xin chao tat ca mng.

- ➔ **Kết quả:** Server quét mảng poll, nhận diện từ khóa all và phát (broadcast) tin nhắn này. Cả hai Client A02 và A03 đều nhận được dòng A01: Xin chao tat ca mng.

Kiểm chứng tính năng Unicast (Gửi đích danh):

+ Client A01 tiếp tục gửi lệnh: A03 hello 3.

+ **Kết quả:** Server trích xuất được ID đích là A03. Ngay lập tức, chỉ duy nhất Client A03 (Góc dưới phải) nhận được tin nhắn A01: hello 3. Client A02 tuyệt đối không nhận được tin nhắn này, chứng minh tính bảo mật và định tuyến chính xác của bộ đệm.

=> Server hoàn toàn không bị nghẽn (block) thao tác khi phải liên tục lắng nghe người mới, nhận tin nhắn, bóc tách chuỗi (parse string) và định tuyến gửi tin cho nhiều Client cùng một lúc.

Bài tập – Telnet Server

Sử dụng hàm select()/poll(), viết chương trình **telnet_server** thực hiện các chức năng sau:

Khi đã kết nối với 1 client nào đó, yêu cầu client gửi user và pass, so sánh với file cơ sở dữ liệu là một file text, mỗi dòng chứa một cặp user + pass ví dụ:

admin admin

guest nopass

...

Nếu so sánh sai, không tìm thấy tài khoản thì báo lỗi đăng nhập.

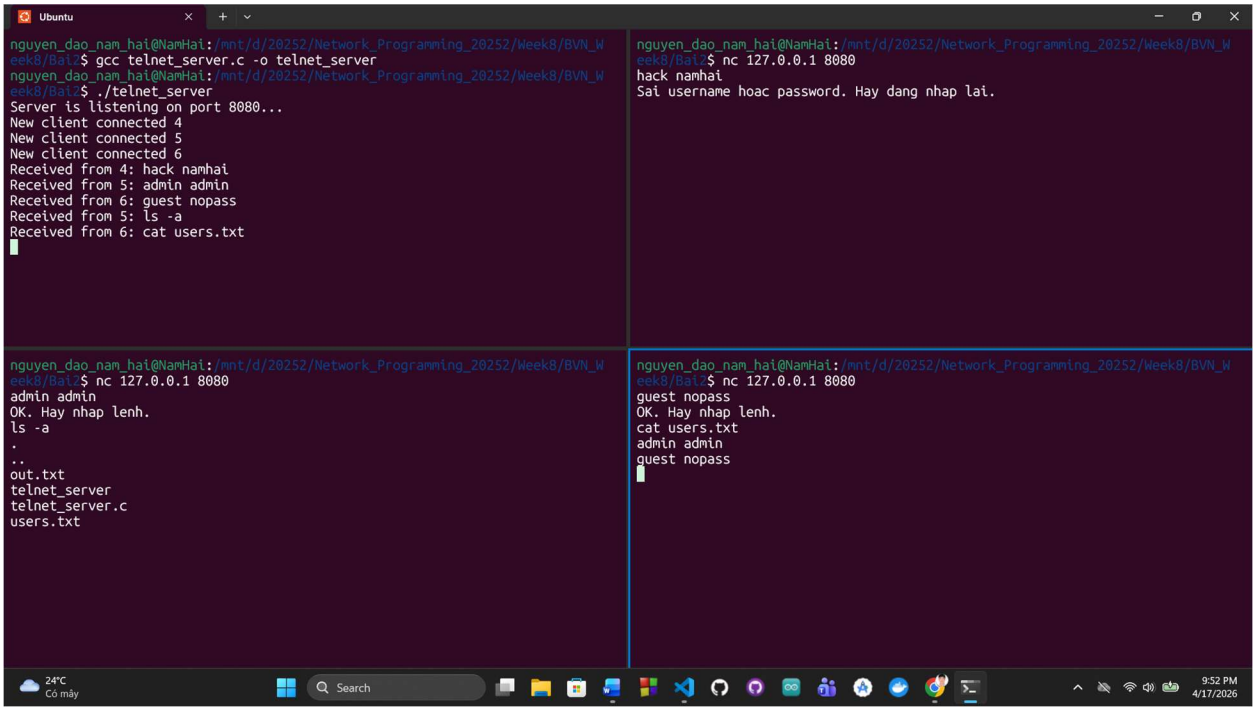
Nếu đúng thì đợi lệnh từ client, thực hiện lệnh và trả kết quả cho client.

Dùng hàm system("dir > out.txt") để thực hiện lệnh.

dir là ví dụ lệnh dir mà client gửi.

> out.txt để định hướng lại dữ liệu ra từ lệnh dir, khi đó kết quả lệnh dir sẽ được ghi vào file văn bản.

Demo:



Giải thích:

Góc trên trái là Server (quản lý kết nối). Ba ô còn lại đóng vai trò là 3 Client riêng biệt kết nối đến Server qua cổng 8080.

Kiểm chứng xử lý lỗi xác thực (Góc trên phải): Client nhập sai tài khoản (hack namhai). Server đối chiếu với file dữ liệu thất bại và lập tức trả về thông báo từ chối: "Sai username hoac password...", yêu cầu nhập lại mà không làm văng chương trình.

Kiểm chứng thực thi lệnh hệ thống 1 (Góc dưới trái): Client đăng nhập thành công với tài khoản hợp lệ (admin admin). Sau đó, Client gửi lệnh ls -a. Server đã nhận lệnh, thực thi qua hàm system(), định hướng I/O ra file out.txt và trả về màn hình Client toàn bộ danh sách các file trong thư mục hiện tại.

Kiểm chứng thực thi lệnh hệ thống 2 (Góc dưới phải): Song song cùng thời điểm, một Client khác đăng nhập thành công bằng guest nopass. Khi Client này gửi lệnh cat users.txt, Server xử lý độc lập và trả về chính xác nội dung của file cơ sở dữ liệu.

Thông qua log của Server (Góc trên trái), có thể thấy cả 3 quá trình (từ chối tài khoản sai, chạy lệnh ls -a, và chạy lệnh cat) đều được tiếp nhận và xử lý trơn tru. Hàm poll() đã điều phối xuất sắc các File Descriptor, đảm bảo luồng thực thi lệnh của Client này không làm nghẽn (block) tiến trình đăng nhập hay chat của Client khác. Kết quả trả về đúng đích, luồng I/O được quản lý chặt chẽ.